G.H. Raisoni International Skill Tech University
Department of Mechanical Engineering

Unit 2 :Stack & Queues

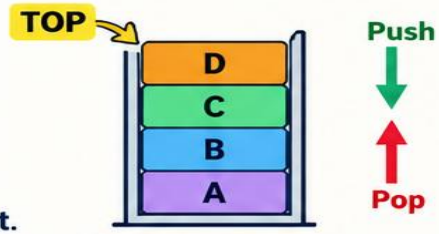
Prof. Sagar Mahajan

STACKS

Last In, First Out — (LIFO)

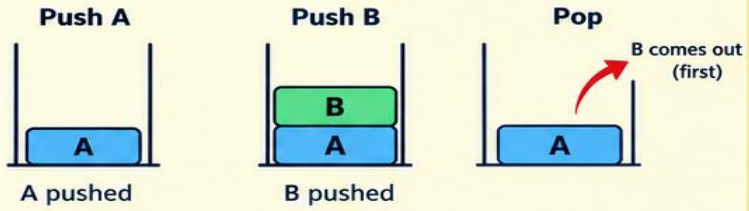
1. What is a Stack? (Concept & LIFO Principle)

A stack is a linear data structure where insertion and deletion happen at **only one end** — called the **TOP**. The last element inserted is the first one to come out. This is called **LIFO — Last In, First Out.**



Stack (Top at one end)

LIFO in Action



Last inserted (B) comes out first → **LIFO**

Mechanical Analogy — The Sheet Metal Stack



The worker places a new sheet on top...



....and when he needs one, he picks the top sheet first.



He cannot pull the bottom sheet without removing the ones above it.

This is exactly how a stack works!

More Mechanical Examples



Spring-loaded rivet magazine in a riveting gun
(last rivet loaded is the first fired).



Vertical tool crib
(top tool is the most accessible).



Piston rings stacked on a mandrel during assembly
(last ring placed is the first removed).



Tower of Hanoi
(a classic stack problem).

Key Terms

TOP



Pointer / index to the most recently inserted element.

Underflow



Trying to pop from an empty stack (like trying to pick a sheet from an empty bin).

Empty stack → Underflow

Overflow



Trying to **push into** a full stack (like trying to add a sheet when the bin is full).

Full stack → Overflow

Same idea... Different world... Stack everywhere !

STACK OPERATIONS
(with Mechanical Analogies)

Same Concept... Real World Examples!

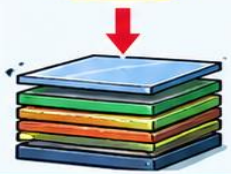
PUSH

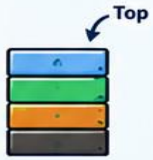
Insert an element at the **TOP**.

Mechanical Mapping



Placing a new sheet on top of the stack in a press shop.





New element is added at the top.

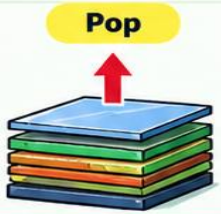
POP

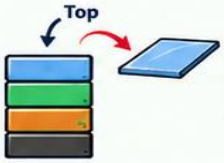
Remove and return the **TOP** element.

Mechanical Mapping



Picking the top sheet for the next operation.





Top element is removed and returned.

PEEK / TOP

View the **TOP** element **WITHOUT** removing it.

Mechanical Mapping



Looking at which sheet is on top — you don't take it, just check.





Just view the top element. No removal.

isEmpty

Check if stack has zero elements.

Mechanical Mapping



Checking if the sheet bin is empty before starting a job.



- ✓ **True** → Empty
- ✗ **False** → Has elements

isFull

Check if stack has reached capacity.

Mechanical Mapping



Checking if the sheet bin is full — no more can be added.



- ✓ **True** → Full (can't add more)
- ✗ **False** → Space available (can add more)

Same Operations... Different World... But the Logic is Always the Same !

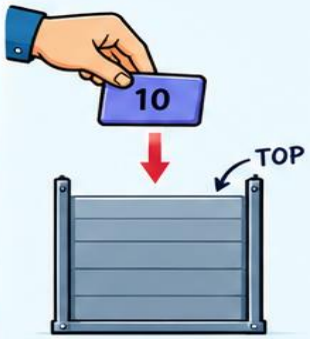
STACK

Step-by-Step Example (Push & Pop)

LIFO
Last In, First Out

1 Push(10)

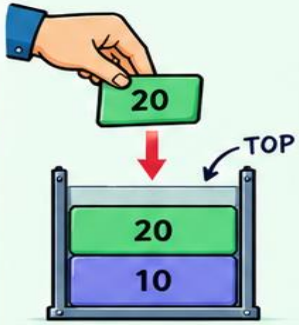
Insert 10 at the TOP.



Stack: [10]

2 Push(20)

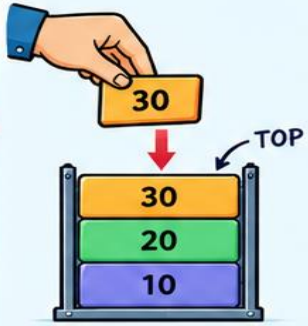
Insert 20 at the TOP.



Stack: [10, 20]

3 Push(30)

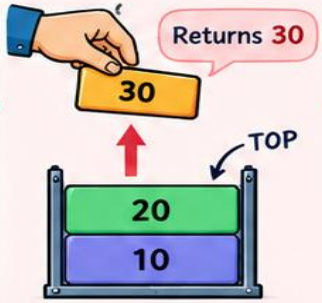
Insert 30 at the TOP.



Stack: [10, 20, 30]

4 Pop()

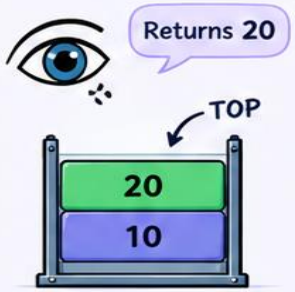
Remove and return the TOP element.



Stack: [10, 20]
(TOP is now 20)

5 Peek()

View the TOP element without removing it.



Stack: [10, 20]
(unchanged)

Visual Summary



Time Complexity

All operations are O(1) — constant time.

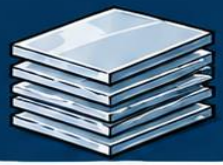
Whether the stack has 5 or 5000 elements, push/pop/peek take the same time.

This is a key placement question!

Key Takeaways

- Stack follows **LIFO** (Last In, First Out).
- Push, Pop, Peek are all O(1).

Simple concept... Powerful in real world!

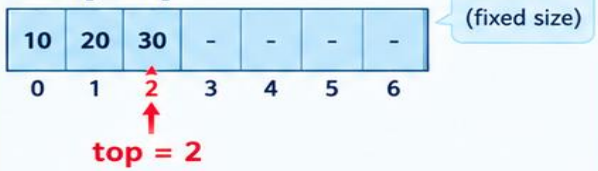


STACK IMPLEMENTATION
(Array-Based Implementation)

Simple Fast Effective

1. Data Structure

- Use a fixed-size array **stack[MAX]**.
- Use an integer **top**, initialized to **−1**.
- Stack grows from left to right.



2. Operations

1 PUSH(value)

```
top = top + 1;
stack[top] = value;
```

2 POP()

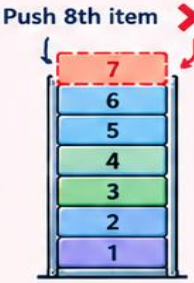
```
value = stack[top];
top = top - 1;
return value;
```

3 PEEK()

```
return stack[top];
```
(without removing)

3. Limitation

⚠ Fixed size → **overflow** if you exceed **MAX**.

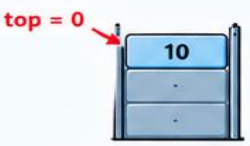


Like a fixed-capacity sheet bin.

Step-by-Step Example

1 Push(10)

```
top = top + 1;
stack[top] = 10;
```



Stack: [10]

2 Push(20)

```
top = top + 1;
stack[top] = 20;
```



Stack: [10, 20]

3 Push(30)

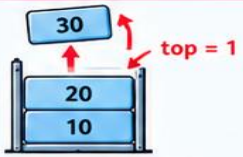
```
top = top + 1;
stack[top] = 30;
```



Stack: [10, 20, 30]

4 Pop()

```
value = stack[top];
top = top - 1;
```
(returns 30)



Stack: [10, 20]
(TOP is now 20)

5 Peek()

```
return stack[top];
```
(returns 20)



Stack: [10, 20]
(unchanged)

Mechanical Mapping



A fixed-capacity rack in a workshop → you can only store as many tools as the rack has slots.

- **Push** → Place tool on top (next empty slot)
- **Pop** → Take top tool (most recent)
- **Peek** → Just look at top tool (don't remove)

Key Takeaways

- Stack is implemented using an array and a **top** index.
- All operations (push, pop, peek) are **O(1)**.
- **Limitation**: fixed size → overflow.
- Mechanical analogy: fixed-capacity rack.

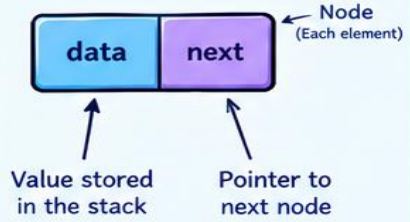
Same idea... Different world... **Stack everywhere !**

STACK IMPLEMENTATION
(Linked List-Based Implementation)
(Conceptual – Detailed in Unit IV)

Dynamic Size No Fixed Limit!

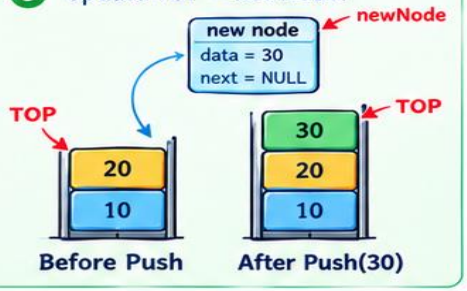
1 Node Structure

Each element is a **node** with two fields:



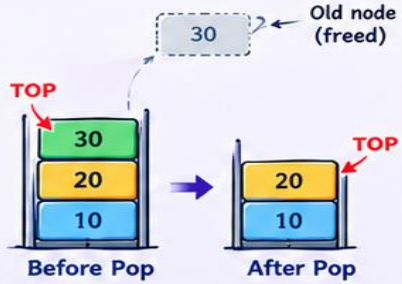
2 Push (Insert)

1. Create a new node.
2. Set **newNode->next = TOP.**
3. Update **TOP = newNode.**



3 Pop (Delete)

1. Move **TOP** to **TOP->next.**
2. Free the old top node.



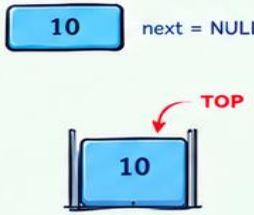
4 Advantage

- **No fixed size** — grows and shrinks dynamically.
- Like a collapsible storage rack that expands as needed.



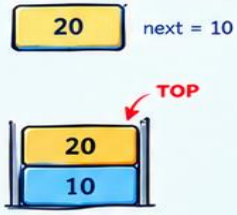
Complete Example

Push(10)



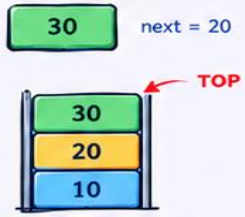
Stack: [10]

Push(20)



Stack: [20, 10]

Push(30)



Stack: [30, 20, 10]

Pop()



Returns: 30
Stack: [20, 10]

Peek()

Returns: 20
(without removing)



Stack: [20, 10]

Conceptual Code (C++-like)

```
struct Node {
    int data;
    Node* next;
};

Node* TOP = nullptr;
```

```
void push(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = TOP;
    TOP = newNode;
}
```

```
void pop() {
    if (TOP == nullptr) return;
    Node* temp = TOP;
    TOP = TOP->next;
    delete temp;
}
```

```
int peek() {
  if (TOP == nullptr) return -1;
  return TOP->data;
}
```

Real World Analogy



A collapsible storage rack that expands as needed.

Just like a stack!

Same Logic... But Different Implementation!

Dynamic • Flexible • Powerful

APPLICATIONS OF STACKS
(with Mechanical Mapping)

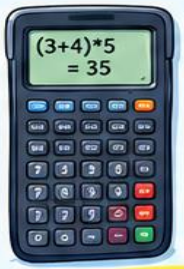
Same Logic... Real World Impact!

1 Expression Evaluation

What It Does:
- Converts / evaluates arithmetic expressions (infix → postfix → result).

Example: (3 + 4) × 5

1. Infix: (3 + 4) × 5
2. Postfix: 3 4 + 5 ×
3. Result: 35



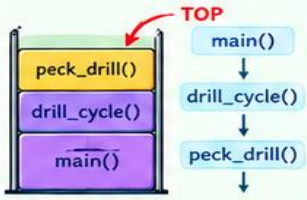
Calculator = Stack in Action

Mechanical Connection:
A scientific calculator used by a mechanical engineer for stress / load calculations.



2 Function Call Management

What It Does:
- OS uses a "call stack" to track function calls.



Call Stack (in OS)

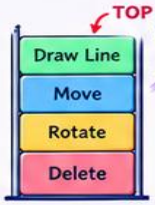
Mechanical Connection:
A CNC controller tracking nested subroutines (e.g., a drilling cycle calling a peck-drilling sub-cycle).



CNC = Stack of Subroutines

3 Undo Operation

What It Does:
- Each action pushed onto a stack; undo pops the last action.



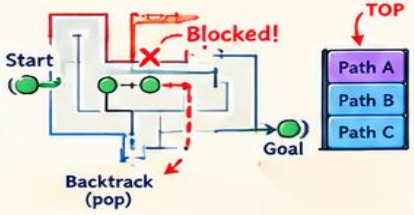
Mechanical Connection:
CAD software (AutoCAD / SolidWorks) undo — the last change you made is the first undone.



CAD = Stack for Undo

4 Backtracking

What It Does:
- Explore a path, push decisions, pop when stuck.



Mechanical Connection:
Robot path planning on a shop floor — try a route, backtrack if blocked.



Robot = Stack of Decisions

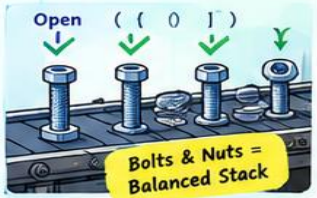
5 Balancing Symbols

What It Does:
- Check if parentheses / braces are balanced.

(a + [b * c]) ✓ Balanced

(a + [b * c]} ✗ Not Balanced

Mechanical Connection:
Checking mating parts — every bolt (open) must have a nut (close) in the correct order.



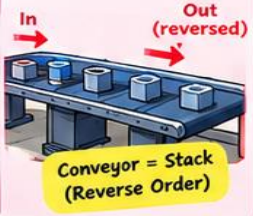
Bolts & Nuts = Balanced Stack

6 Reversing Data

What It Does:
- Push all, then pop all → reversed order.



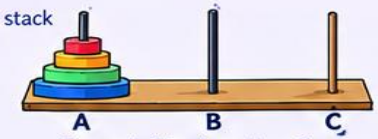
Mechanical Connection:
Reversing a conveyor — parts go in one order, come out reversed.



Conveyor = Stack (Reverse Order)

7 Tower of Hanoi

What It Does:
- Classic recursion / stack problem.



Move all disks from A → C (only top disk can be moved).

Mechanical Connection:
Physical disk-and-peg puzzle — literally a mechanical stack.



Disks = Stack (Real World)

Why It Matters?

- ✓ Used in compilers, OS, calculators, CAD, robotics, industrial automation.
- ✓ Helps solve real-world problems with simple logic.
- ✓ Same core idea → Stack!

Simple Concept... Big Impact!

Stacks... From Calculations to Robots — Everywhere!

Think Stack... Solve Real Problems!

Three Notations

Same expression — Different order — Same meaning!

Notation	Example	Where Operator Sits
1 Infix	A + B	Operator between operands (how humans write)
2 Prefix (Polish)	+ A B	Operator before operands
3 Postfix (Reverse Polish)	A B +	Operator after operands (how computers evaluate)

Mechanical Mapping

Think of it as three ways to write a manufacturing sequence:

Infix: Drill THEN Tap
(operator in middle)



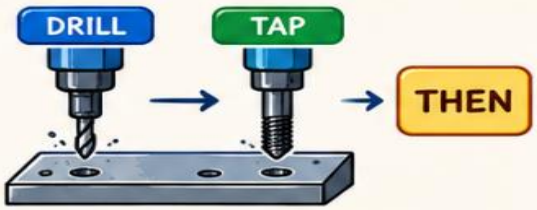
Drill THEN Tap

Prefix: THEN Drill Tap
(operator first)



THEN Drill Tap

Postfix: Drill Tap THEN
(operator last)



Drill Tap THEN

Same operation ✓ Different order ✓ Same result!

Infix to Postfix Conversion (using Stack)

Algorithm (simple rules):

1. Scan the infix expression left to right.
2. If operand (A, B, C...) → output it directly.
3. If '(' → push onto stack.
4. If ')' → pop from stack and output until '(' is found; discard '('.
5. If operator (+, −, *, /, ^) → pop from stack and output all operators with higher or equal precedence; then push the current operator.
6. At the end, pop and output all remaining operators.

Precedence:

^ >	* = /	> + = −
(Highest)	(Medium)	(Lowest)

Example:

Convert **A + B * C** to postfix.

Infix: A + B * C → **Postfix:** A B C * +

Step-by-Step Conversion

Step	Symbol	Stack (Top → Bottom)	Output
1	A	(empty)	A
2	+	+ (top)	A
3	B	+	A B
4	*	* (top), +	A B
5	C	* (top), +	A B C
6	End	(pop all)	A B C * +

Visual Walkthrough

1. Read A → Output: A (no stack operation)
2. Read + → Push to stack



→ Output: A

4. Read * → Push to stack (higher precedence than +, so no pop)



→ Output: A B

6. End → Pop all



Output: A B C * +

Result: A B C * +

✓ Simple rules ✓ Stack helps
✓ Works for any expression

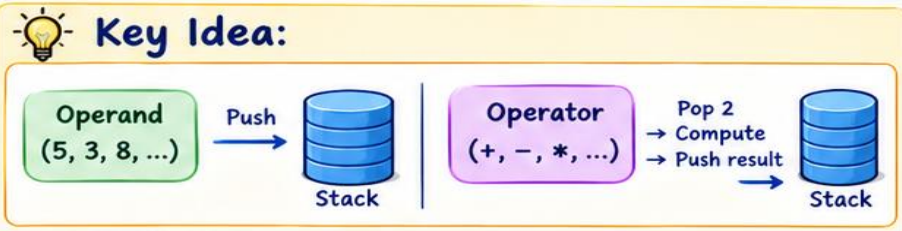


Postfix Evaluation (using Stack)

Algorithm:

1. Scan postfix left to right.
2. If operand → push onto stack.
3. If operator → pop two operands, apply operator, push result back.
4. At end, the single value on stack is the answer.

Key Idea:



Example Expression:

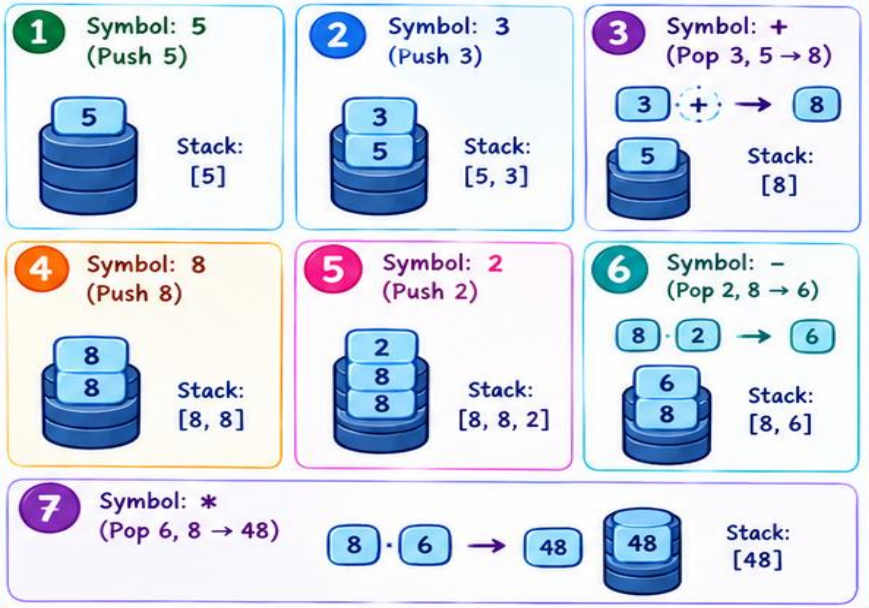
5 3 + 8 2 − *

Worked Example: 5 3 + 8 2 − *

Step	Symbol	Action	Stack (top → bottom)
1	5	Push 5	[5]
2	3	Push 3	[5, 3]
3	+	Pop 3, 5 → 5+3=8, push 8	[8]
4	8	Push 8	[8, 8]
5	2	Push 2	[8, 8, 2]
6	−	Pop 2, 8 → 8−2=6, push 6	[8, 6]
7	*	Pop 6, 8 → 8×6=48, push 48	[48]

Answer: 48

Visual Step-by-Step



Mechanical Mapping:

Postfix evaluation is like a CNC program where you first load tools (push operands), then execute operations (pop and compute), and the final result is the finished part dimension.



QUEUES with Mechanical Mapping) (Concept & FIFO Principle)

First In... First Out!

1. What is a Queue?

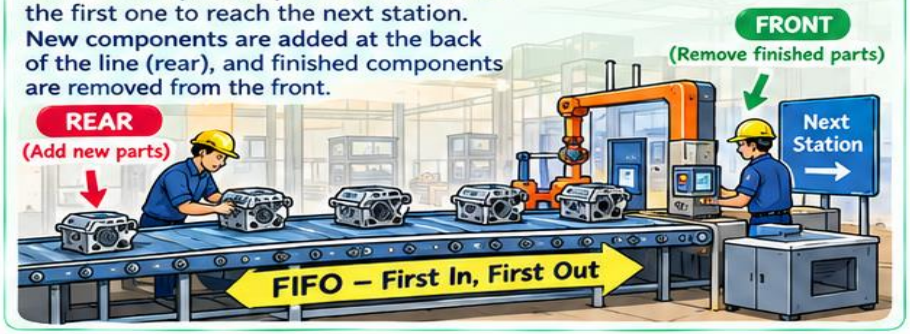
Definition (Simplest Form):
A queue is a linear data structure where insertion happens at one end (REAR) and deletion happens at the other end (FRONT).

The first element inserted is the first one to come out — **FIFO** (First In, First Out).

Key Idea:
REAR → Enqueue (Add)
FRONT → Dequeue (Remove)

2. Mechanical Analogy – The Assembly Line

On a conveyor belt in an assembly line, the first component placed on the belt is the first one to reach the next station. New components are added at the back of the line (rear), and finished components are removed from the front.



3. Queue Visual (Basic Structure)



Queue: [10, 20, 30, 40]

First In (10) → First Out (10)

4. More Mechanical Examples

1. CNC Machine Jobs



First job submitted is machined first (FCFS scheduling).

2. Toll Plaza Vehicles



First vehicle in line pays first.

3. Heat Treatment Batch



First batch loaded is first processed.

4. Raw Material Queue



First-arrived stock is first-consumed.

5. Print Queue



First document sent is first printed.

5. Key Terms

FRONT
Index/pointer to the element that will be dequeued next.



REAR
Index/pointer to the most recently enqueued element.



Underflow
Dequeue from an empty queue (empty conveyor belt — nothing to pick).



Overflow
Enqueue into a full queue (belt is full — no space to place another part).



6 Visual Summary

Queue = FIFO (First In, First Out)



- Linear structure
- Insertion at REAR
- Deletion at FRONT
- Used everywhere in real world!

Like a line... Everyone gets their turn!

Queues... Simple Concept ... Powerful in Real World!

From Assembly Lines to Operating Systems... Queues Make it Happen!

FIFO (First In First Out)

Queue Operations

(with Mechanical Analogies)

Like a conveyor belt... first in, first out!

Operation	What It Does	Mechanical Mapping
ENQUEUE (Insert at REAR)	Insert an element at the REAR.	 Placing a new component at the back of the conveyor belt.
DEQUEUE (Remove from FRONT)	Remove and return the FRONT element.	 Picking the finished component from the front of the belt.
FRONT / PEEK	View the FRONT element without removing it.	 Looking at which component is next to be picked — you don't take it, just check.
REAR	View the last inserted element.	 Looking at the most recently placed component at the back.
isEmpty	Check if queue has zero elements.	 Checking if the conveyor belt is empty.
isFull	Check if queue has reached capacity.	 Checking if the conveyor belt is full — no space for more parts.

Queue in Action (Visual Overview)



- ✓ FIFO order
- ✓ Insert at rear
- ✓ Remove from front
- ✓ Used in scheduling, buffering, and more!

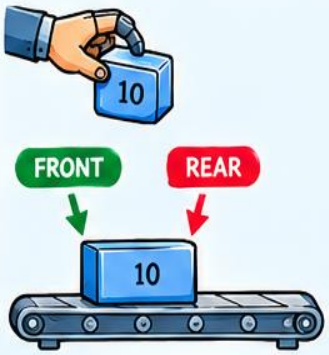


QUEUE — Step-by-Step Example (Enqueue & Dequeue)

First In... First Out!

1 Enqueue(10)

Add 10 to the rear.

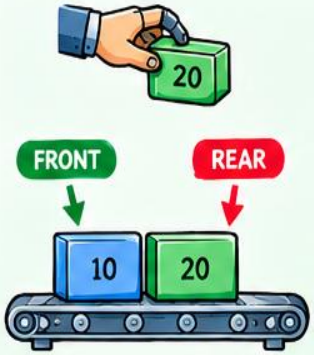


Queue: [10]

FRONT = 10, REAR = 10

2 Enqueue(20)

Add 20 to the rear.

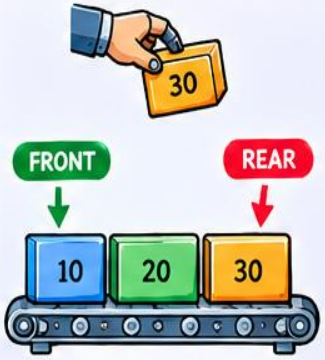


Queue: [10, 20]

FRONT = 10, REAR = 20

3 Enqueue(30)

Add 30 to the rear.

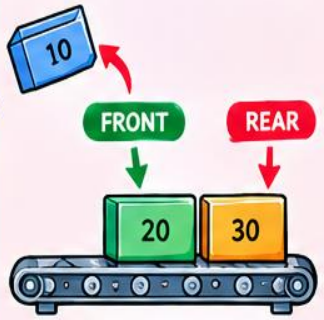


Queue: [10, 20, 30]

FRONT = 10, REAR = 30

4 Dequeue()

Remove and return the front element (10).



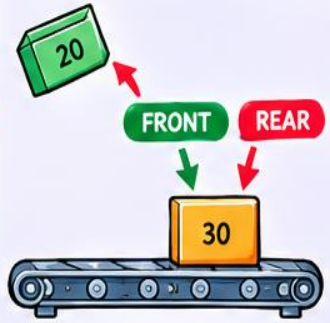
Queue: [20, 30]

FRONT = 20, REAR = 30

(Returned: 10)

5 Dequeue()

Remove and return the front element (20).

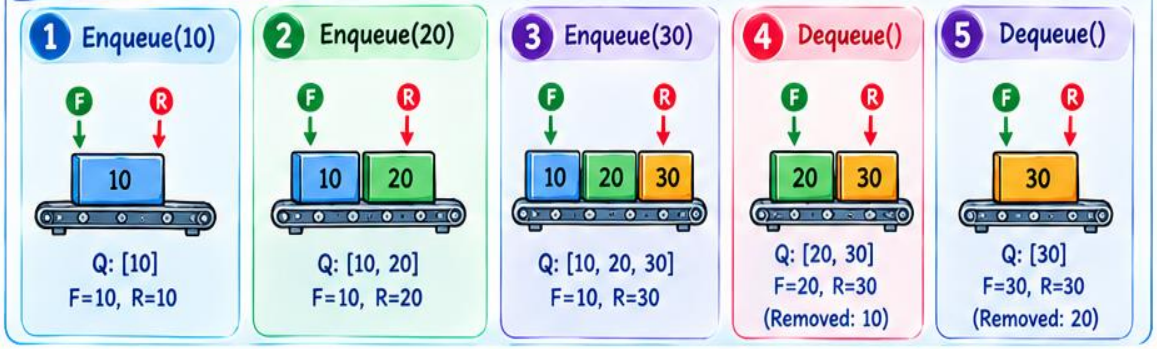


Queue: [30]

FRONT = 30, REAR = 30

(Returned: 20)

Visual Summary



Key Terms

- **FRONT** → Index/pointer to the element that will be dequeued next.
- **REAR** → Index/pointer to the most recently enqueued element.
- **Underflow** → Dequeue from an empty queue (empty conveyor belt – nothing to pick).
- **Overflow** → Enqueue into a full queue (belt is full – no space to place another part).

Time Complexity

Enqueue and Dequeue are both **O(1)** in a properly implemented queue (circular or linked).

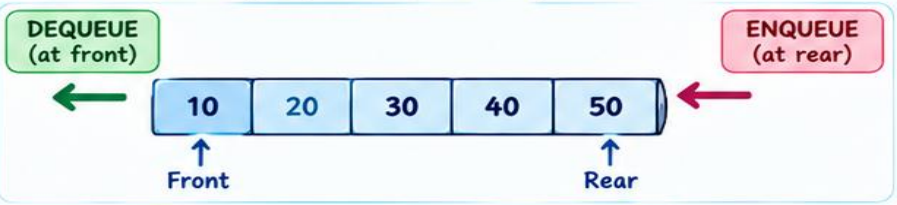
A common placement question!

Types of Queues

Same concept...
Different shapes...
Different uses!

(a) Simple / Linear Queue

- Elements are arranged in a straight line.
- ENQUEUE at rear, DEQUEUE at front.
- **Problem:** After several dequeues, front moves forward and space at the beginning is wasted (false overflow / drifting queue).
- **Mechanical mapping:** Straight conveyor belt.



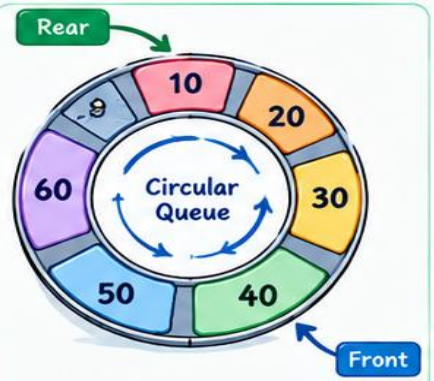


Mechanical Mapping:
Straight conveyor belt — once parts move past a point, empty space at the start cannot be reused unless the belt is reset.

Problem:
- False overflow / Drifting queue

(b) Circular Queue

- Last position connects back to the first (forms a circle).
- Solves the false overflow problem.
- **Implementation:** rear = (rear + 1) % MAX
- **Mechanical mapping:** Rotary conveyor / carousel.



Key Formulas (for exams):

Front	=	(front + 1) % MAX (on dequeue)
Rear	=	(rear + 1) % MAX (on enqueue)
Full	:	(rear + 1) % MAX == front
Empty	:	front == -1



Mechanical Mapping:
Rotary conveyor / carousel — every slot gets reused. No wasted space.

(c) Priority Queue

- Each element has a priority.
- **Highest priority is dequeued first** (regardless of insertion order).
- **Mechanical mapping:** CNC job scheduling / emergency repair jobs.
- **Implementation:** Heap (later) or sorted array/linked list.

Order of Dequeue (by priority):

P1 (High) 1	P3 (Medium) 3	P5 (Medium) 4	P2 (Low) 5	P4 (Low) 8

← First Last →

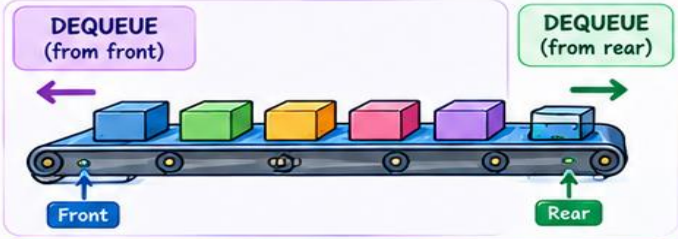
(Priority: 1 = highest, 8 = lowest)



Mechanical Mapping:
Job scheduling on a CNC machine — a rush order (high priority) jumps ahead of regular jobs. Or emergency repair jobs get priority over routine servicing.

(d) Double-Ended Queue (Deque)

- Insertion and deletion can happen at **both** ends (front and rear).
- Mechanical mapping: Two-way conveyor.
- Useful in flexible manufacturing systems (FMS).





Mechanical Mapping:
Two-way conveyor — parts can be loaded or unloaded from either end. Useful in FMS where routing is dynamic.

Variants:

1. Input-restricted deque (insert only at rear, delete from both ends).
2. Output-restricted deque (insert at both ends, delete only from front).

Queue Implementation

(a) Array-Based Implementation (Simple & Circular)

- Use an array to store queue elements.
- Two types: **Simple (Linear)** and **Circular Queue**

1 Simple Queue (Linear)

Initial State: front = 0, rear = -1

0	1	2	3	4
-	-	-	-	-

(empty)

Operations:

Enqueue → rear++

Dequeue → front++

2 Circular Queue

Initial State:

front = 0, rear = -1

0	1	2	3	4
-	-	-	-	-

(last position connects to first)

Operations:

Enqueue → rear = (rear + 1) % MAX

Dequeue → front = (front + 1) % MAX

Key Conditions:

- Full: (rear + 1) % MAX == front
- Empty: front == -1

Examples

1 Simple Queue (Linear) – Example

After Enqueue (10, 20, 30)

0	1	2	3	4
10	20	30	-	-

front (0), rear (2)

→ After Dequeue (remove 10)

0	1	2	3	4
-	20	30	-	-

front (1), rear (2)

2 Circular Queue – Example

After Enqueue (10, 20, 30, 40)

0	1	2	3	4
10	20	30	40	-

front (0), rear (3)

→ After Enqueue (50) – wraps around

0	1	2	3	4
50	20	30	40	10

front (1), rear (0)

⚠ Limitation

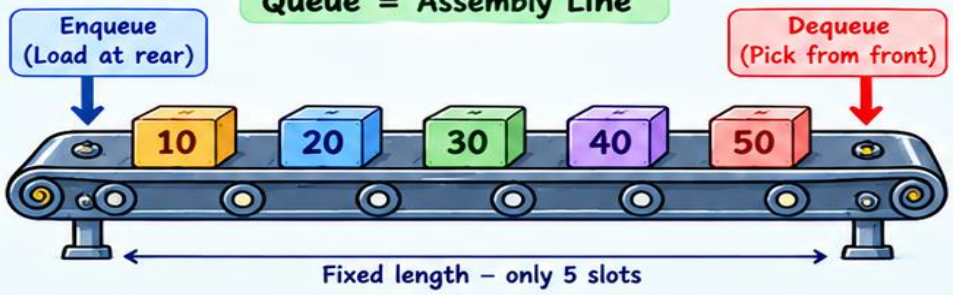
- Fixed size array → overflow if full.
- Wastes space in simple queue (false overflow).

0	1	2	3	4
10	20	30	40	50

Queue Full!

2 Mechanical Mapping – Fixed-Length Assembly Line

Queue = Assembly Line



Real-World Analogy

- A fixed-length assembly line.
- Parts are added at the rear (enqueue).
- Parts are removed from the front (dequeue).
- If the line is full, upstream must stop (no more parts can be added).



Fixed-length conveyor = Fixed-size queue

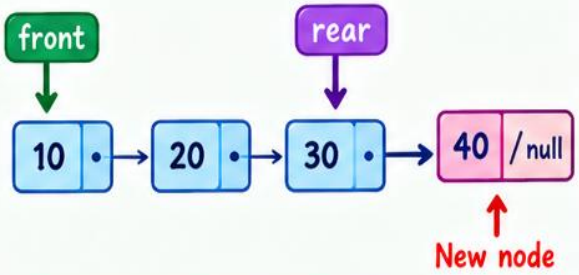
(b) Linked List-Based Implementation

(Conceptual — Detailed in Unit IV)

Key Points

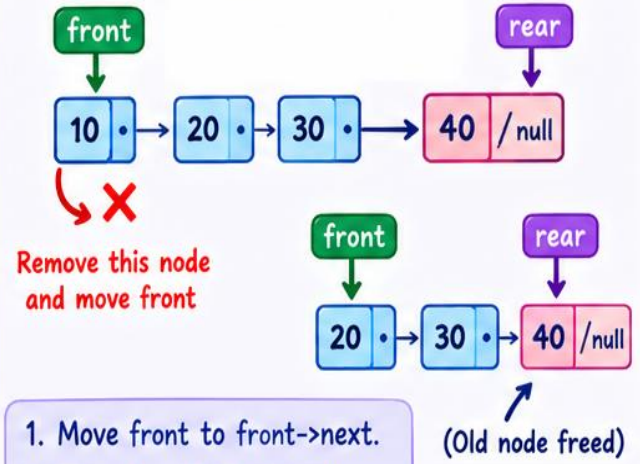
- Maintain two pointers: **front** and **rear**
- **Enqueue:** Create a new node, attach it after rear, update rear.
- **Dequeue:** Move front to front->next, free the old front node.

Enqueue (Insert at Rear)



1. Create new node (e.g., 40).
2. Attach it after rear.
3. Update rear.

Dequeue (Remove from Front)

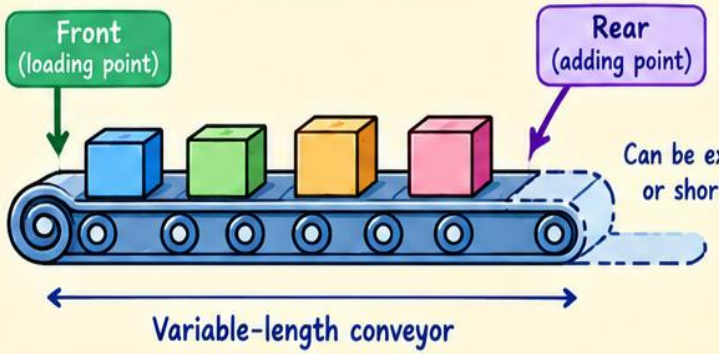


1. Move front to front->next.
2. Free the old front node.

Advantage

- No fixed size — grows/shrinks dynamically.
- Like a **variable-length conveyor** that can be extended or shortened as production demands change.

Mechanical Analogy



Just like a flexible assembly line that can grow or shrink as per the need.

Applications of Queues
(with Mechanical Mapping)

1 Job Scheduling (FCFS)

First-Come-First-Served scheduling of jobs on a machine.

Job Queue



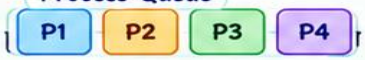


CNC machine job queue — jobs machined in order of arrival.

2 CPU Scheduling

OS schedules processes in a queue.

Process Queue





Scan Cycle
1. Read Inputs
2. Execute Tasks
3. Update Outputs

PLC scan cycle — inputs processed in order, tasks queued.

3 Printer / Spooler Queue

Documents printed in order.

Print Queue





Job card / traveler printing in a shop floor office.

4 Assembly Line Simulation

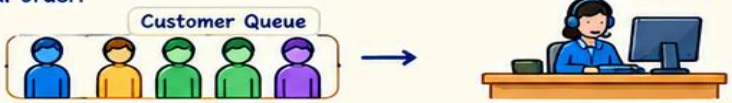
Model parts flowing through stations.



Conveyor belt — first part in, first part out.

5 Call Center / Customer Queue

Customers served in arrival order.



Service desk at a machine shop — first customer served first.

6 Traffic Signal Management

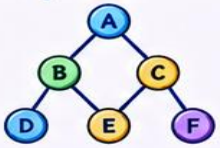
Vehicles queued at a signal.



Toll plaza / signal — first vehicle in line goes first.

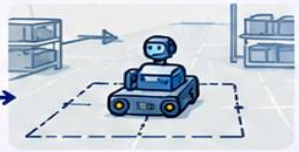
7 Breadth-First Search (BFS)

Graph traversal using a queue.



Queue

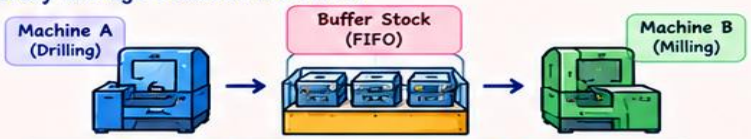




Robot exploration on a shop floor — explore nearest nodes first (covered in Unit V).

8 Buffer in Production

Temporary storage between machines.



Buffer stock between two machining stations — first-in-first-out flow.

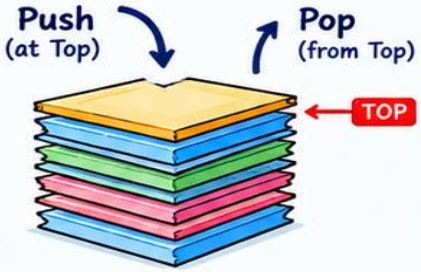
Queues keep things moving — in machines, in systems and in our daily life!

Stack vs Queue — Side by Side

Same Goal — Different Order!

Stack

LIFO (Last In, First Out)



Mechanical Analogy:



Stack of sheets / rivet magazine

Operations

Push, Pop, Peek

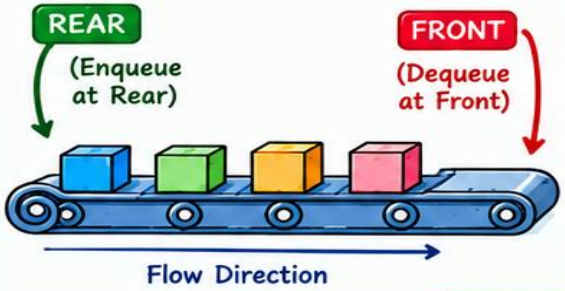
Real Mechanical Use



CAD undo, CNC subroutine calls

Queue

FIFO (First In, First Out)



Mechanical Analogy:



Conveyor belt / assembly line

Operations

Enqueue, Dequeue, Front, Rear

Real Mechanical Use



Job scheduling, buffer stock, assembly line

Feature	Stack	Queue
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion end	TOP (only one end)	REAR
Deletion end	TOP (same end)	FRONT (opposite end)
Mechanical analogy	Stack of sheets / rivet magazine	Conveyor belt / assembly line
Operations	Push, Pop, Peek	Enqueue, Dequeue, Front, Rear
Time complexity	O(1) for all ops	O(1) for all ops (circular/linked)
Primary use	Reversal, backtracking, expression evaluation	Order preservation, scheduling, buffering
Real mechanical use	CAD undo, CNC subroutine calls	Job scheduling, buffer stock, assembly line

Different order. Same purpose — Organize and manage data efficiently!


Learn
Explore
Grow
Thank You !
Good Concepts Build Great Engineers
Better Students = Brighter Future
Prof. Sagar Mahajan Sir.
Learn
Practice
Understand
Grow
YouTube Channel :
Sir Sagar Mahajan
Computer Engineering
DSA & Algorithms
Computer Architecture
Discrete Mathematics
Programming & Beyond
Keep Learning
Concepts
Coding
Problem Solving
Career Growth
Stay Connected & Keep Learning !